

Laboratorium Ilmu Rekayasa dan Komputasi

TEKNIK INFORMATIKA
STEI ITB

IF2224 - Formal Language and
Automata Theory

PASCAL-S Compiler

Dokumen Terpusat

Daftar Revisi

[DD-MM-YYYY] Revisi X. Keterangan Revisi.

[09-10-2025] Release Milestone 1.

Daftar Isi

Daftar Revisi.....	2
Daftar Isi.....	3
I. Pendahuluan.....	4
II. Teori Singkat.....	6
III. Deskripsi Tugas.....	8
IV. Pesan Asisten.....	9
Referensi.....	10
Lampiran.....	11

I. Pendahuluan

Bahasa pemrograman merupakan sarana utama bagi manusia untuk menuliskan algoritma dan instruksi yang dapat dijalankan oleh komputer. Melalui bahasa pemrograman tingkat tinggi, pengembang perangkat lunak dapat mengekspresikan logika pemecahan masalah dengan cara yang lebih mudah dipahami manusia. Agar instruksi-instruksi ini dapat dijalankan di perangkat keras, diperlukan *compiler* (penyusun) yang menerjemahkan kode sumber tingkat tinggi tersebut menjadi kode tingkat rendah (*machine code*) tanpa mengubah fungsionalitas program. Dengan adanya kompiler, program yang ditulis dalam bahasa tingkat tinggi dapat dieksekusi oleh mesin komputer. Selain itu, kompiler berperan penting dalam mengoptimasi program serta memastikan program tersebut bebas dari kesalahan-kesalahan dasar, sehingga menghasilkan aplikasi yang efisien, stabil, dan aman untuk dijalankan. Peran inilah yang membuat kompiler menjadi komponen fundamental dalam ekosistem pengembangan perangkat lunak modern.

Proses kompilasi pada umumnya terdiri dari beberapa tahap bertingkat. Tahap awal yang sangat krusial adalah *analisis leksikal* (lexical analysis). Pada tahap ini, teks kode sumber dibaca karakter demi karakter oleh suatu komponen yang disebut *lexer* atau *scanner*, kemudian diuraikan menjadi unit-unit bermakna terkecil yang disebut *token*. Token-token tersebut mewakili elemen-elemen leksikal bahasa pemrograman, seperti kata kunci (*keywords*), pengenalan (*identifiers*), literal, dan simbol operator. Hasil keluaran dari analisis leksikal ini berupa deretan token yang akan diteruskan ke tahap berikutnya, yaitu analisis sintaksis, untuk diperiksa struktur dan tata bahasanya. Dengan demikian, analisis leksikal berfungsi mempersiapkan data mentah kode sumber menjadi bentuk yang lebih terstruktur, sehingga memudahkan *parser* dalam membangun pohon sintaks abstrak dari program yang dikompilasi.

Tugas besar mata kuliah IF2224 Teori Bahasa Formal dan Otomata yang berjudul **"Pascal-S Compiler"** berfokus pada implementasi tahap analisis leksikal tersebut. Bahasa yang digunakan, Pascal-S, merupakan varian subset (subhimpunan) dari bahasa pemrograman Pascal yang disederhanakan untuk tujuan pendidikan pembuatan sistem kompilasi. Tujuan utama dari tugas ini adalah membangun sebuah *scanner* untuk bahasa Pascal-S yang mampu mengenali setiap token pada kode sumber Pascal-S sesuai dengan aturan leksikal yang telah

ditetapkan. Scanner tersebut akan dibuat dengan menerapkan teknik analisis leksikal berbasis *DFA* (Deterministic Finite Automaton), yakni menggunakan model *automata hingga deterministik* sebagai mesin pengenalan pola token. Pendekatan ini sejalan dengan prinsip fundamental dalam implementasi penganalisis leksikal, di mana proses *scanning* dapat dimodelkan sebagai mesin *finite state yang* berubah state mengikuti aliran karakter input dan mengidentifikasi token-token valid ketika mencapai state akhir yang menerima. Dengan kata lain, *scanner* Pascal-S yang dibangun akan merealisasikan DFA yang membaca program sumber Pascal-S dan memetakan rangkaian karakter menjadi token-token sesuai definisi tata bahasa leksikalnya.

Pembentukan *scanner* berbasis DFA pada tugas ini menunjukkan relevansi langsung antara teori dan praktik dalam lingkup Teori Bahasa Formal dan Otomata. Dalam teori bahasa formal, token-token dalam bahasa pemrograman umumnya digambarkan oleh *regular expression* atau tata bahasa regular, yang selanjutnya dapat direpresentasikan dan dikenali menggunakan model automata hingga (finite automata). Mahasiswa telah mempelajari bahwa setiap grammar regular dapat dikonversi menjadi ekspresi regular, yang kemudian dapat diubah menjadi *automata hingga nondeterministik* (NFA) dan akhirnya disederhanakan menjadi *automata hingga deterministik* (DFA) tanpa kehilangan daya pengenalnya. Konsep inilah yang diterapkan dalam proyek Pascal-S Compiler: kumpulan token Pascal-S diperlakukan sebagai sebuah *bahasa regular yang* dikenali oleh DFA. Dengan mengembangkan *scanner* Pascal-S berbasiskan DFA, tugas ini memperlihatkan bagaimana model teoritis automata digunakan secara nyata untuk memecahkan permasalahan praktis dalam pembangunan kompiler. Hal ini menegaskan keterkaitan erat antara konsep-konsep pada Teori Bahasa Formal dan Otomata – seperti bahasa regular, grammar, dan mesin otomat – dengan implementasinya dalam teknologi penyusun (*compiler*). Melalui implementasi scanner Pascal-S ini, mahasiswa diharapkan dapat lebih memahami bagaimana teori automata deterministik digunakan untuk mengenali pola bahasa pemrograman, sekaligus merasakan pentingnya dasar teori tersebut dalam merancang sistem *software* yang kompleks seperti kompiler.

II. Teori Singkat

Proses kompilasi pada dasarnya terdiri dari beberapa tahapan yang saling berhubungan. Setiap tahapan menghasilkan keluaran yang akan menjadi masukan bagi tahapan berikutnya. Oleh karena itu, penting untuk memastikan bahwa hasil dari setiap langkah sudah benar sebelum melanjutkan ke langkah selanjutnya.

Dalam tugas ini digunakan mekanisme kompilator yang mencakup lima tahap utama, yaitu **Lexical Analysis (Lexer)**, **Syntax Analysis (Parser)**, **Semantic Analysis**, **Intermediate Code Generation**, dan **Interpreter**. Berikut penjelasan singkat mengenai masing-masing tahap tersebut.

II.1. Lexical Analysis

Tahapan ini bertujuan untuk mengubah source code Pascal-S dari kumpulan karakter mentah menjadi unit-unit bermakna yang disebut *token*. Setiap token memiliki jenis (type) dan nilai (value), serta akan menjadi masukan bagi tahap berikutnya dalam proses kompilasi.

Masukan	Kode Pascal-S (.pas)
Keluaran	List Token (misalnya VAR(X), ASSIGN(=), OPERATOR(+))
Otomata	Deterministic finite automata (DFA)

Contoh input kode PASCAL-S
<pre> program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end. </pre>
Contoh output <i>list of tokens</i>
<pre> KEYWORD(program) IDENTIFIER(Hello) </pre>

```

SEMICOLON (;)
KEYWORD (var)
IDENTIFIER (a)
COMMA (,)
IDENTIFIER (b)
COLON (:)
KEYWORD (integer)
SEMICOLON (;)
KEYWORD (begin)
IDENTIFIER (a)
ASSIGN_OPERATOR (:=)
NUMBER (5)
SEMICOLON (;)
IDENTIFIER (b)
ASSIGN_OPERATOR (:=)
IDENTIFIER (a)
ARITHMETIC_OPERATOR (+)
NUMBER (10)
SEMICOLON (;)
IDENTIFIER (writeln)
LPARENTHESIS ( ( )
STRING_LITERAL ('Result = ')
COMMA (,)
IDENTIFIER (b)
RPARENTHESIS ( ) )
SEMICOLON (;)
KEYWORD (end)
DOT (.)

```

II.2. Syntax Analysis

Tahapan ini bertujuan untuk menganalisis struktur sintaksis dari list of tokens yang dihasilkan lexer dan membangun **Parse Tree** yang merepresentasikan hierarki struktur program Pascal-S sesuai dengan grammar bahasa. Parse Tree ini akan menjadi masukan bagi tahap semantic analysis dalam proses kompilasi.

Masukan	List Token (misalnya VAR(X), ASSIGN(=), OPERATOR(+))
Keluaran	Parse Tree
Algoritma	Recursive Descent

Berikut merupakan daftar node yang mungkin muncul pada hasil Parse Tree.

No	Nama Node	Deskripsi	Aturan Produksi	Contoh
1	<program>	Node root yang merepresentasikan keseluruhan program Pascal-S	program $\rightarrow$ program-header + declaration-part + compound-statement + DOT	Seluruh struktur program dari awal hingga akhir
2	<program-header>	Bagian kepala program yang berisi nama program	KEYWORD(program) + IDENTIFIER + SEMICOLON	program Hello;
3	<declaration-part>	Bagian deklarasi yang dapat berisi const, type, var, procedure, function	(const-declaration)* + (type-declaration)* + (var-declaration)* + (subprogram-declaration)*	Semua deklarasi sebelum "mulai"
4	<const-declaration>	Deklarasi konstanta	KEYWORD(konstanta) + (IDENTIFIER = value + SEMICOLON)+	konstanta MAX = 100;
5	<type-declaration>	Deklarasi tipe data baru	KEYWORD(tipe) + (IDENTIFIER = type-definition + SEMICOLON)+	tipe Range = 1..10;
6	<var-declaration>	Deklarasi variabel	KEYWORD(variabel) + (identifier-list + COLON + type + SEMICOLON)+	variabel a, b: integer;
7	<identifier-list>	Daftar identifer	IDENTIFIER	a, b, c

	ist>	yang dipisahkan koma	(COMMA + IDENTIFIER)*	
8	<type>	Tipe data (integer, real, boolean, char, array)	KEYWORD(integer /real/boolean/char) atau array-type	integer, larik[1..10] dari integer
9	<array-type>	Definisi tipe array	KEYWORD(larik) + LBRACKET + range + RBRACKET + KEYWORD(dari) + type	larik[1..10] dari integer
10	<range>	Rentang nilai untuk array atau subrange	expression + RANGE_OPERATOR(..) + expression	1..10, 'a'..'z'
11	<subprogram-declaration>	Deklarasi prosedur atau fungsi	procedure-declaration atau function-declaration	prosedur print(x: integer);
12	<procedure-declaration>	Deklarasi prosedur	KEYWORD(prosedur) + IDENTIFIER + (formal-parameter-list)? + SEMICOLON + block + SEMICOLON	Prosedur dengan parameter opsional
13	<function-declaration>	Deklarasi fungsi	KEYWORD(fungsi) + IDENTIFIER + (formal-parameter-list)? + COLON + type + SEMICOLON + block + SEMICOLON	Fungsi yang mengembalikan nilai
14	<formal-param	Daftar parameter	LPARENTHESIS +	(x, y:

	eter-list>	formal	parameter-group (SEMICOLON + parameter-group) * + RPARENTHESIS	integer; z: real)
15	<compound-statement>	Blok statement yang diawali begin dan diakhiri end	KEYWORD(mulai) + statement-list + KEYWORD(selesai)	mulai ... selesai
16	<statement-list>	Daftar statement yang dipisahkan semicolon	statement (SEMICOLON + statement)*	Urutan statement dalam blok
17	<assignment-statement>	Statement penugasan nilai ke variabel	IDENTIFIER + ASSIGN_OPERATOR(:=) + expression	x := 5, y := x + 10
18	<if-statement>	Statement kondisional	KEYWORD(jika) + expression + KEYWORD(maka) + statement + (KEYWORD(lainnya) + statement)?	jika x > 0 maka y := 1 lainnya y := 0
19	<while-statement>	Perulangan dengan kondisi di awal	KEYWORD(selama) + expression + KEYWORD(kerjakan) + statement	selama x < 10 kerjakan x := x + 1
20	<for-statement>	Perulangan dengan counter	KEYWORD(untuk) + IDENTIFIER + ASSIGN_OPERATOR + expression + (KEYWORD(ke)/KEYWORD(turun-ke)) + expression + KEYWORD(kerjakan) + statement	untuk i := 1 ke 10 kerjakan ...
21	<procedure-call>	Pemanggilan	IDENTIFIER +	writeln('Hell

	ll>	prosedur	(LPARENTHESIS + parameter-list + RPARENTHESIS)?	o'), print(x, y)
22	<parameter-list>	Daftar parameter aktual saat pemanggilan	expression (COMMA + expression)*	'Result = ', b, 100
23	<expression>	Ekspresi yang menghasilkan nilai	simple-expression (relational-operator + simple-expression)?	x + y, a > b, 5 * (x + 1)
24	<simple-expression>	Ekspresi tanpa operator relasional	(ARITHMETIC_OPERATOR(+/-))? term (additive-operator + term)*	x + y - z, 5 * 2
25	<term>	Bagian ekspresi dengan prioritas lebih tinggi	factor (multiplicative-operator + factor)*	x * y, a bagi b
26	<factor>	Unit terkecil dalam ekspresi	IDENTIFIER / NUMBER / CHAR_LITERAL / STRING_LITERAL / (LPARENTHESIS + expression + RPARENTHESIS) / LOGICAL_OPERATOR(tidak) + factor / function-call	x, 5, 'a', (x+y), flag "tidak"
27	<function-call>	Pemanggilan fungsi yang mengembalikan nilai	IDENTIFIER + LPARENTHESIS + (parameter-list)? + RPARENTHESIS	sqrt(16), max(a, b)
28	<relational-operator>	Operator	=, <, <=, >, >=	x = 5, y > 10

	perator>	perbandingan		
29	<additive-operator>	Operator penjumlahan/pengurangan	+, -, atau	$x + y$, a atau b
30	<multiplicative-operator>	Operator perkalian/pembagian	*, /, bagi, mod, dan	$x * y$, a bagi b, p dan q

Program harus menggunakan Recursive Descent dengan **grammar yang di-hardcode dalam program** dan keseluruhan grammar dijelaskan kembali dalam laporan yang dibuat.

Input awal dari program **tetap merupakan source code PASCAL-S** sehingga sebelum dimasukkan ke parser, kode dimasukkan terlebih dahulu ke lexer.

Input Command (contoh yang digunakan menggunakan Python) Format: python [Compiler] [Kode Pascal]
python compiler.py program.pas
Isi program.pas
<pre> program Hello; variabel a, b: integer; mulai a := 5; b := a + 10; writeln('Result = ', b); selesai.</pre>
program.pas ketika diubah menjadi token
<pre> KEYWORD(program) IDENTIFIER(Hello) SEMICOLON(;) KEYWORD(variabel) IDENTIFIER(a) COMMA(,) IDENTIFIER(b)</pre>

```

COLON(:)
KEYWORD(integer)
SEMICOLON(;)
KEYWORD(mulai)
IDENTIFIER(a)
ASSIGN_OPERATOR(:=)
NUMBER(5)
SEMICOLON(;)
IDENTIFIER(b)
ASSIGN_OPERATOR(:=)
IDENTIFIER(a)
ARITHMETIC_OPERATOR(+)
NUMBER(10)
SEMICOLON(;)
KEYWORD(writeln)
LPARENTHESIS((
STRING_LITERAL('Result = ')
COMMA(,)
IDENTIFIER(b)
RPARENTHESIS())
SEMICOLON(;)
KEYWORD(selesai)
DOT(.)

```

Proses (grammar belum tentu benar)

```

program → program-header declaration-part compound-statement DOT
program-header → KEYWORD(program) IDENTIFIER SEMICOLON
declaration-part → ...
compound-statement → ...
... grammar lainnya

```

Berdasarkan grammar "program", non-terminal yang pertama ditemukan adalah "program-header". Setelah itu di dalam "program-header" dilakukan pengecekan token input.

```

KEYWORD(program)
IDENTIFIER>Hello)
SEMICOLON(;)

```

Karena token input sesuai, berarti "program-header" valid. Sehingga output sementara akan seperti ini

```
<program>
```

```

├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(Hello)
│   └── SEMICOLON(;)

```

... sisa output

Kemudian sisa pengecekan dilakukan untuk "declaration-part", "compound-statement", dan DOT.

Kemudian juga disini dilakukan **error checking** di level parser. Contohnya adalah misal untuk "program-header", token yang dibaca adalah seperti berikut.

```

KEYWORD(program)
IDENTIFIER(Hello)
DOT(.)

```

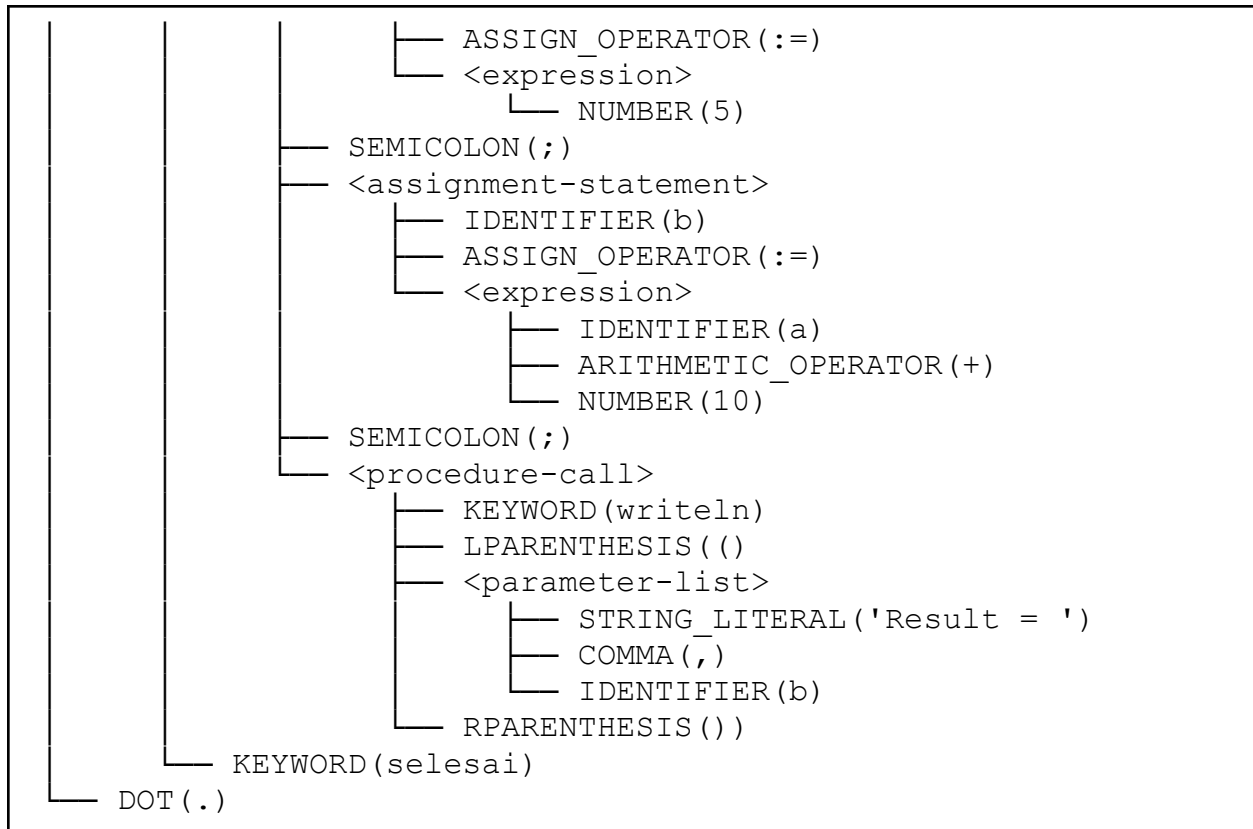
Maka parser akan mengeluarkan error. Untuk pesan error dibebaskan yang penting informatif. Contohnya adalah "Syntax error: unexpected token DOT(.), expected SEMICOLON(;)".

Keluaran (output)

```

<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER(Hello)
│   └── SEMICOLON(;)
├── <declaration-part>
│   └── <var-declaration>
│       ├── KEYWORD(variabel)
│       └── <var-list>
│           ├── <identifier-list>
│           │   ├── IDENTIFIER(a)
│           │   ├── COMMA(,)
│           │   └── IDENTIFIER(b)
│           ├── COLON(:)
│           ├── <type>
│           │   └── KEYWORD(integer)
│           └── SEMICOLON(;)
├── <compound-statement>
│   ├── KEYWORD(mulai)
│   └── <statement-list>
│       └── <assignment-statement>
│           └── IDENTIFIER(a)

```



- II.3. Semantic Analysis **[TBA]**
- II.4. Intermediate Code Generation **[TBA]**
- II.5. Interpreter **[TBA]**

III. Deskripsi Tugas

Untuk tugas ini, mahasiswa dituntut untuk membuat program kompilar bahasa PASCAL-S yang terdiri dari lima tahapan yang telah dijelaskan pada bagian sebelumnya. Per tahapannya akan dikerjakan secara terpisah dan ketentuan untuk setiap tahapannya dapat dibaca pada dokumen berikut ini:

Tahapan	Laman Spesifikasi	Tanggal Tenggat
Lexical Analysis (Lexer)	 Milestone 1 - Tubes I...	Minggu, 19 Oktober 2025 Pukul 23.59 WIB
Syntax Analysis (Parser)	 Spesifikasi Milestone ...	Minggu, 16 November 2025 Pukul 23.59 WIB
Semantic Analysis	[TBA]	[TBA]
Intermediate Code Generation	[TBA]	[TBA]
Interpreter	[TBA]	[TBA]

Selain itu juga terdapat beberapa ketentuan umum yang berlaku untuk pengerjaan semua milestone, yaitu:

- *Deadline* pengumpulan bersifat **HARD DEADLINE**.
- Diharuskan membuat **laporan** untuk **masing-masing milestone-nya** yang ketentuannya dijelaskan pada setiap dokumen spesifikasi.
- **Penilaian** akan dilakukan **per pengumpulan milestone**. Pastikan kode yang dikumpulkan pada setiap milestone-nya bisa dijalankan dan juga berikan README yang memuat cara menjalankannya dengan lengkap dan jelas.
- **Komponen penilaian** akan dijelaskan pada setiap dokumen spesifikasi.
- Apabila diperlukan, mahasiswa juga diperbolehkan untuk mengadakan **asistensi** dengan asisten terpilih mereka. Untuk teknis asistensinya diserahkan kepada mahasiswa sendiri untuk berkoordinasi langsung dengan asisten mereka masing-masing.

IV. Pesan Asisten

“Enjoy your stay, my dear guests”

– Ikhwan –

“Elite TBFO knowledge”

– Eka –

“No fear, Just like Tom Cruise”

– Farhan –

“If you don’t go, you’ll never know.”

– Ciko –

“Semester 6 mau diasistenin IRK lagi ga hehehe”

– Scifo –

“👍”

– Rafi –

Referensi

"PASCAL-S: A Subset and its implementation"

<http://pascal.hansotten.com/uploads/pascals/PASCAL-S%20A%20subset%20and%20its%20Implementation%20012.pdf>

"Compiler Design - Lexical Analysis"

https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm

"Introduction to Automata Theory, Languages, and Computation. 3rd Edition"

https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf

Lampiran

Laman Pembagian Kelompok: [📅 Pembagian Kelompok - Tubes IF2224 TBFO](#)

Laman QnA Tugas Besar: [📅 QNA - Tubes IF2224 TBFO](#)

Form Pengumpulan: [Form Pengumpulan - Tubes IF2224 TBFO](#)

Laman Spesifikasi Milestone 1: [📅 Spesifikasi Milestone 1 - Tubes IF2224 TBFO](#)